

Aluna: Gabriela Queiroz Ribeiro
Turma: 3º informática

Configuração do Ambiente

Abra o seu terminal e execute os seguintes passos para garantir que você tem o Java instalado.

1. Atualizar repositórios e instalar o JDK

```
maquinavirtual@maquinavirtual-VirtualBox:~$ sudo apt update
sudo apt install default-jdk -y
[sudo] senha para maquinavirtual:
Atingido:1 http://br.archive.ubuntu.com/ubuntu noble InRelease
Obter:2 http://br.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Obter:3 http://br.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
```

2. Verificar a instalação

```
maquinavirtual@maquinavirtual-VirtualBox:~$ java -version
openjdk version "21.0.9" 2025-10-21
OpenJDK Runtime Environment (build 21.0.9+10-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.9+10-Ubuntu-124.04, mixed mode, sharing)
maquinavirtual@maquinavirtual-VirtualBox:~$ javac -version
javac 21.0.9
```

Implementação dos Códigos

Crie uma pasta para organizar o projeto e entre nela:

```
maquinavirtual@maquinavirtual-VirtualBox:~$ mkdir pratica_sockets
maquinavirtual@maquinavirtual-VirtualBox:~$ cd pratica_sockets
maquinavirtual@maquinavirtual-VirtualBox:~/pratica_sockets$
```

Crie 4 arquivos abaixo dentro desta pasta utilizando o nano.

```
maquinavirtual@maquinavirtual-VirtualBox:~/pratica_sockets$ sudo nano TCPServer.java
maquinavirtual@maquinavirtual-VirtualBox:~/pratica_sockets$ sudo nano TCPClient.java
maquinavirtual@maquinavirtual-VirtualBox:~/pratica_sockets$ sudo nano UDPServer.java
maquinavirtual@maquinavirtual-VirtualBox:~/pratica_sockets$ sudo nano UDPClient.java
```

1. Servidor TCP (TCPServer.java) - Máquina virtual

```

import java.io.*;
import java.net.*;
import java.util.Base64;

public class TCPServer {
    public static void main(String[] args) {
        int port = 12345;
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("--> Servidor TCP ouvindo na porta " + port);
            while (true) {
                // Aguarda conexão
                Socket socket = serverSocket.accept();
                System.out.println("Cliente TCP conectado: " + socket.getInetAddress());

                // Streams de entrada e saída
                BufferedReader input = new BufferedReader(new InputStreamReader(socket.getInputStream()));
                PrintWriter output = new PrintWriter(socket.getOutputStream(), true);

                String message = input.readLine();
                if (message != null) {
                    System.out.println("Msg Recebida: " + message);
                    // "Criptografia" simples usando Base64
                    String encrypted = Base64.getEncoder().encodeToString(message.getBytes());
                    output.println(encrypted);
                    System.out.println("Resposta enviada.");
                }
                socket.close();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

2. Cliente TCP (TCPClient.java) - Máquina real

```

import java.io.*;
import java.net.*;

public class TCPClient {
    public static void main(String[] args) {
        String hostname = "200.18.140.239";
        int port = 12345;

        try (Socket socket = new Socket(hostname, port)) {
            PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader input = new BufferedReader(new InputStreamReader(socket.getInputStream()));

            String message = "Ola Mundo TCP";
            output.println(message);
            System.out.println("Enviado: " + message);

            String response = input.readLine();
            System.out.println("Recebido (Criptografado): " + response);
        } catch (IOException e) {
            System.out.println("Erro na conexão TCP: " + e.getMessage());
        }
    }
}

```

3. Servidor UDP (UDPServer.java) - Máquina virtual

```

import java.net.*;
import java.util.Base64;

public class UDPServer {
    public static void main(String[] args) {
        int port = 9876;
        try (DatagramSocket socket = new DatagramSocket(port)) {
            byte[] receiveBuffer = new byte[1024];
            byte[] sendBuffer;

            System.out.println("--> Servidor UDP ouvindo na porta " + port);

            while (true) {
                DatagramPacket receivePacket = new DatagramPacket(receiveBuffer, receiveBuffer.length);
                socket.receive(receivePacket); // Bloqueia até receber pacote

                String message = new String(receivePacket.getData(), 0, receivePacket.getLength());
                System.out.println("  Msg UDP Recebida: " + message);

                String encrypted = Base64.getEncoder().encodeToString(message.getBytes());
                sendBuffer = encrypted.getBytes();

                // Pega endereço de quem mandou para responder
                InetAddress IPAddress = receivePacket.getAddress();
                int clientPort = receivePacket.getPort();

                DatagramPacket sendPacket = new DatagramPacket(sendBuffer, sendBuffer.length, IPAddress, clientPort);
                socket.send(sendPacket);
                System.out.println("  Resposta UDP enviada.");
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

4. Cliente UDP (UDPClient.java) - Máquina real

```

public class UDPClient {
    public static void main(String[] args) {
        String hostname = "200.18.140.239";
        int port = 9876;

        try (DatagramSocket socket = new DatagramSocket()) {
            socket.setSoTimeout(2000);
            InetAddress IPAddress = InetAddress.getByName(hostname);
            byte[] sendData;
            byte[] receiveData = new byte[1024];

            String message = "Ola Mundo UDP";
            sendData = message.getBytes();

            DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, port);
            socket.send(sendPacket);
            System.out.println("Enviado: " + message);

            DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
            socket.receive(receivePacket);

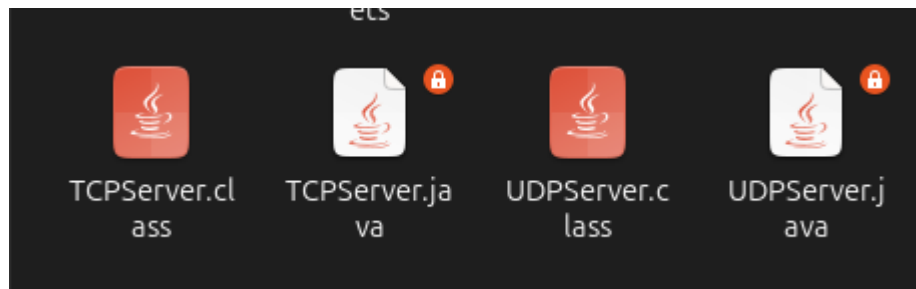
            String response = new String(receivePacket.getData(), 0, receivePacket.getLength());
            System.out.println("Recebido (Criptografado): " + response);
        } catch (SocketTimeoutException e) {
            System.out.println("Erro: Tempo de espera esgotado (Pacote perdido ou bloqueado pelo Firewall).");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

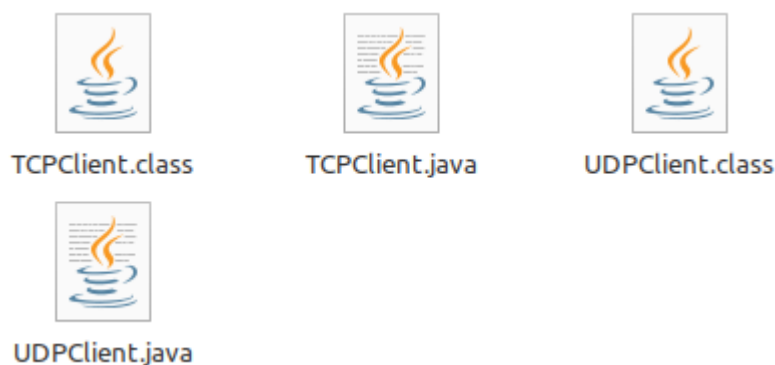
Por fim, compile todos os arquivos de uma vez:

```
maquinavirtual@maquinavirtual-VirtualBox:~$ javac *.java
maquinavirtual@maquinavirtual-VirtualBox:~$
```

Virtual:



Real:



Execução e Testes de Firewall

Inicie o Servidor TCP e UDP.

```
maquinavirtual@maquinavirtual-VirtualBox:~$ java TCPServer
Servidor TCP rodando na porta 12345...
```

```
maquinavirtual@maquinavirtual-VirtualBox:~$ java UDPServer
--> Servidor UDP ouvindo na porta 9876
```

Como cliente:

```
cefet@LAB-6201B-05:~/Downloads$ java UDPClient
Enviado: Ola Mundo UDP
Recebido (Criptografado): T2xhIE11bmRvIFVEUA==
cefet@LAB-6201B-05:~/Downloads$
```

```
maquinavirtual@maquinavirtual-VirtualBox:~$ sudo iptables -A INPUT -p tcp --dport 12345 -j DROP
[sudo] senha para maquinavirtual:
maquinavirtual@maquinavirtual-VirtualBox:~$ java TCPClient
```

3.2 Desabilitar UDP e Habilitar TCP

```
maquinavirtual@maquinavirtual-VirtualBox:~$ sudo iptables -F
maquinavirtual@maquinavirtual-VirtualBox:~$ sudo iptables -A INPUT -p tcp --dpor
t 12345 -j ACCEPT
maquinavirtual@maquinavirtual-VirtualBox:~$ sudo iptables -A INPUT -p udp --dpor
t 9876 -j DROP
maquinavirtual@maquinavirtual-VirtualBox:~$ java UDPClient
Enviando pacote UDP...
ERRO UDP: Tempo esgotado! O pacote foi perdido ou bloqueado pelo Firewall.
maquinavirtual@maquinavirtual-VirtualBox:~$ java TCPClient
Enviando: Teste TCP Mensagem
Recebido do Servidor: VGVzdGUgVENQIE1lbnNhZ2Vt
maquinavirtual@maquinavirtual-VirtualBox:~$ █
```